

Midterm – Creating Images with Diffusion Models

Reflection Journal

ITAI 2376 | Team DataProxy

Dataset: Fashion-MNIST

Team members

Elijah Raines • Brandon Matias • Justin Davis
Jonah Joseph • Fifehanmi Ogedengbe • Samantha Mireles

Team Reflection

As Team DataProxy, this project helped us understand how a diffusion model learns to create images by predicting noise. We used Fashion-MNIST, which contains 28×28 grayscale clothing images in 10 classes. We normalized the images to $[-1, 1]$ and split the 60,000 samples into 48,000 for training and 12,000 for validation. Seeing a batch shape of $[64, 1, 28, 28]$ helped us connect the code to the actual images going into the U-Net. The process made more sense when we understood that we already knew the noise we added, so we could compare it with the model's prediction. Our training loss started around 0.1873, and validation loss dropped from about 0.1529 at Epoch 1 to 0.1145 at Epoch 10. That gave us evidence that the model was learning to predict noise more accurately, although we still needed to check the quality of its generated clothing images.

The main challenges were small code mistakes that stopped later parts of the notebook. In Step 4, the noise visualization failed with 'NameError: x_t is not defined' because the forward diffusion function had not defined the noisy image before returning it. Fixing the equation that combines the clean image with random noise allowed that part to work. The visualization then helped show how the clothing became harder to recognize as noise increased. In Step 7, we faced a different problem: the error said 'Expected size 10 but got size 1' at a U-Net skip connection. The class mask had been given the shape of the 10-class vector instead of the expected $[\text{batch}, 1]$ shape. The follow-up correction changed how `c_mask` was created, but the PDF still shows the error rather than a successful rerun. These examples taught us to check both variable definitions and tensor shapes, and to trace a problem back through the inputs instead of only looking at the line where it crashed.

Working together showed us how much each section depended on the others. The setup and data preparation had to be correct before the model, training, and generation sections could use them. Seed 42 helped keep runs more consistent while debugging, and validation and checkpoint saving helped us track and preserve training progress. We also learned to be clear about what our results actually showed: the captured training continued into Epoch 11 of the planned 30, and the CLIP section contained evaluation setup without final scores. If we did this again, we would test a small batch and a short generation run earlier, check the conditioning shapes before a long run, and save the output after each fix. Overall, adding noise to real

clothing images, training the U-Net to predict it, and attempting to reverse the process over 100 steps gave us a much clearer understanding of diffusion than reading the explanation alone.